

InstructGraph 中文精读导读：用“类代码”格式让大模型读懂图、少胡说

原文：Jianing Wang, Junda Wu, Yupeng Hou, Yao Liu, Ming Gao, Julian McAuley (ACL 2024 Findings)

本导读为面向初学者的中文学习笔记：含要点翻译、公式精讲与图表解读。

论文卡片

标题	InstructGraph: Boosting Large Language Models via Graph-centric Instruction Tuning and Preference Alignment
作者	Jianing Wang, Junda Wu, Yupeng Hou, Yao Liu, Ming Gao, Julian McAuley
机构	华东师范大学、加州大学圣地亚哥分校 (UC San Diego)
会议	ACL 2024 Findings (计算语言学协会年会 · 发现专辑)
arXiv	2402.08785
代码	https://github.com/wjn1996/InstructGraph
方向	图 (Graph) × 大模型 × 后训练 (指令微调 + 偏好对齐)
一句话	把所有图都写成统一的“类代码”格式喂给大模型，先指令微调学会图任务，再用 DPO 偏好对齐压幻觉，最终在图推理与图生成上超过 GPT-4。

1 一、这篇论文在解决什么问题

大语言模型 (LLM, 如 GPT、LLaMA) 天生只会处理“一串文字”。但现实世界里很多重要信息天生是图：知识图谱、符号图、社交网络，甚至人脑里隐式的“思维图”。这些图由**节点** (实体) 和**边** (关系) 组成，很难用一段平铺直叙的文字说清楚。于是大家都想把“图”塞进 LLM。作者把已有做法归为两条路、并各指出一个痛点：

1. **“图描述”路线**：人工写一大堆模板，把图“翻译”成自然语言句子喂给 LLM。痛点：模板要海量人工设计；更麻烦的是，LLM 生成的回答**很难再被解析回一张真正的图**。
2. **“图嵌入”路线**：额外训练一个图神经网络 (GNN)，把图编码成向量，注入 LLM 的部分参数里。痛点：把一张大图或复杂图压成向量会**丢信息**，而且也难以把输出还原成图。

在这两条路之外，作者还点出两个更本质的挑战：(1) 图与文本之间存在语义鸿沟，直接拼起来 LLM 未必能真正理解；(2) LLM 在图任务上爱胡说（幻觉）——比如图里缺关键信息时硬答、或者生成一张事实错误 / 自相矛盾 / 信息缺失的图。作者把后者称为图幻觉（graph hallucination）问题。

本文的目标：提出一个叫 **InstructGraph** 的框架，分两步走——先用**图指令微调**（graph instruction tuning）教会 LLM 做各种图任务，再用**图偏好对齐**（graph preference alignment, **DPO 风格**）压低图任务上的幻觉。关键招数是：不外挂任何图编码器，而是用一种统一的“类代码”**结构化格式**把所有图写出来——因为现在的 LLM 本来就很擅长读写代码。

【小白补丁】为什么“把图写成代码”是个好主意

LLM 在海量代码上预训练过，对 `list = [...]`、`(a -> b)` 这类结构化写法天然敏感。把一张图写成 `entity_list=[...]`；`triple_list=[(...)]`；这种“类代码”，既保留了图的完整结构（不丢信息），又落在 LLM 最擅长的格式里，输出还能被程序直接解析回一张图。一举解决了前面两条老路的通病。

2 二、摘要中译

当前的大语言模型（LLM）通过参数更新能否更好地求解图推理与图生成任务？本文提出 **InstructGraph**，一个通过**指令微调**（instruction tuning）与**偏好对齐**（preference alignment）赋予 LLM 图推理与图生成能力的框架。具体而言，我们首先提出一种**结构化格式表述器**（structured format verbalizer），把所有图数据统一成一种通用的“类代码”格式，从而无需任何外部图专用编码器即可简洁地表示图。其次，我们引入**图指令微调**阶段，引导 LLM 求解图推理与图生成任务。最后，我们识别出图任务中潜在的幻觉问题，并采样负样本进行偏好对齐，目标是提升模型输出的可靠性。在多个以图为中心的任务上的大量实验表明，**InstructGraph** 能取得最佳性能，分别超过 GPT-4 与 LLaMA2 达 13% 和 38% 以上。

3 三、核心贡献

1. **提出结构化格式表述器**：用统一的“类代码”格式表示任意图，不外挂图编码器，既不丢结构信息，又对齐 LLM 最擅长的代码能力，输出可被解析回真正的图。
2. **构建图指令微调语料与流程**：把 29 个图任务整理成约 160 万条指令数据，定义**四大类图任务**（图结构建模、图语言建模、图生成建模、图思维建模），统一用标准的因果语言建模目标继续训练 LLM。
3. **提出图偏好对齐缓解幻觉**：针对图推理与图生成设计多种“图幻觉”场景（不实图、冲突图、缺失图等），自动负采样造“错误答案”，用 DPO 优化 LLM 提升可靠性。

4. **大量实验验证**:在图指令任务、图偏好任务以及通用 NLP 任务上全面评测,超过同规模 LLaMA2/Vicuna,并在总体上超过 GPT-4。

4 四、预备知识（小白先看这里）

【小白补丁】什么是指令微调（instruction tuning）

就是把“任务说明 + 输入 + 标准答案”整理成大量样本，继续训练 LLM，让它学会“听懂指令并照做”。本文把各种图任务（判断连通、生成知识图谱、回答图问题……）都写成统一格式的指令样本，用最普通的“预测下一个词”目标来训练，就能让一个通用 LLM 变成会做图任务的 InstructGraph-INS。

【小白补丁】什么是 DPO（直接偏好优化）与偏好对齐

DPO（Direct Preference Optimization）是一种让模型“学会偏好好答案、嫌弃坏答案”的训练方法，是 RLHF（基于人类反馈的强化学习）的轻量替代品：它不需要单独训练奖励模型、也不跑强化学习，只要给一对“（好答案 Y^+ ，坏答案 Y^- ）”，直接用一个公式把 LLM 往“多给 Y^+ 、少给 Y^- ”的方向推。本文的妙处在于：好答案就是原本的正确答案，坏答案靠“故意把图改错”自动生成，不用人工标注。

形式化记号（看不懂可跳过，用到时再回看）：设有 M 个图任务 $D = \{D_1, \dots, D_M\}$ ，每个任务的数据集为 $D_j = \{(I_i, G_i, P_i, A_i)\}_{i=1}^{N_j}$ ，其中 I_i 是指令， P_i 是可选的文本段落， A_i 是最终答案；图 $G_i = (E_i, R_i, T_i, S_i)$ 由节点（实体）集 E_i 、可选关系集 R_i 、边（三元组）集 T_i 和可选文本属性集 S_i 组成。关键的“图 $\rightarrow$ 文字”转换由结构化格式表述器 $M(\cdot)$ 完成：

$$C_i = M(G_i). \quad (*)$$

也就是把整张图 G_i 写成一段“类代码”序列 C_i ，例如 `entity_list=[...]; triple_list=[(...)];`。带文本属性的节点用“面向对象”的写法表达，例如把节点 User1 的评论写成 `User1.review=The film is nice.`。这样所有图都被统一成同一种格式，可以和普通文字无缝拼接喂给 LLM。

5 五、四大类图任务与数据集

如图 1 所示，作者把图相关任务归成四大类（前两类偏图推理，第三类是图生成，第四类兼有推理和生成）。这套分类是全文的骨架，先记住它再看后面会顺很多。

表 1: InstructGraph 的四大类图任务（整理自原文 Figure 1 与 Table 1）。

任务大类	包含的子任务	做什么
图结构建模 (Graph Structure Modeling)	连通检测、环检测、哈密顿路径、二分图匹配、最短路径、度数计算	让 LLM 理解最基础的图结构（只给节点、有/无向边、可选权重）。纯图推理
图语言建模 (Graph Language Modeling)	图标题生成、图问答、节点分类、链接预测、相关性检测、协同过滤	让 LLM 同时理解图的结构与语义并回答问题。纯图推理
图生成建模 (Graph Generation Modeling)	知识图谱生成、结构图生成	给一段文字, 让 LLM 生成一张“类代码”格式的图。图生成
图思维建模 (Graph Thought Modeling)	算术、符号、机器人、逻辑推理	三步走：找问题主体 生成一张“思维图”表达推理过程 据图给出答案。推理 + 生成，仅用于评测

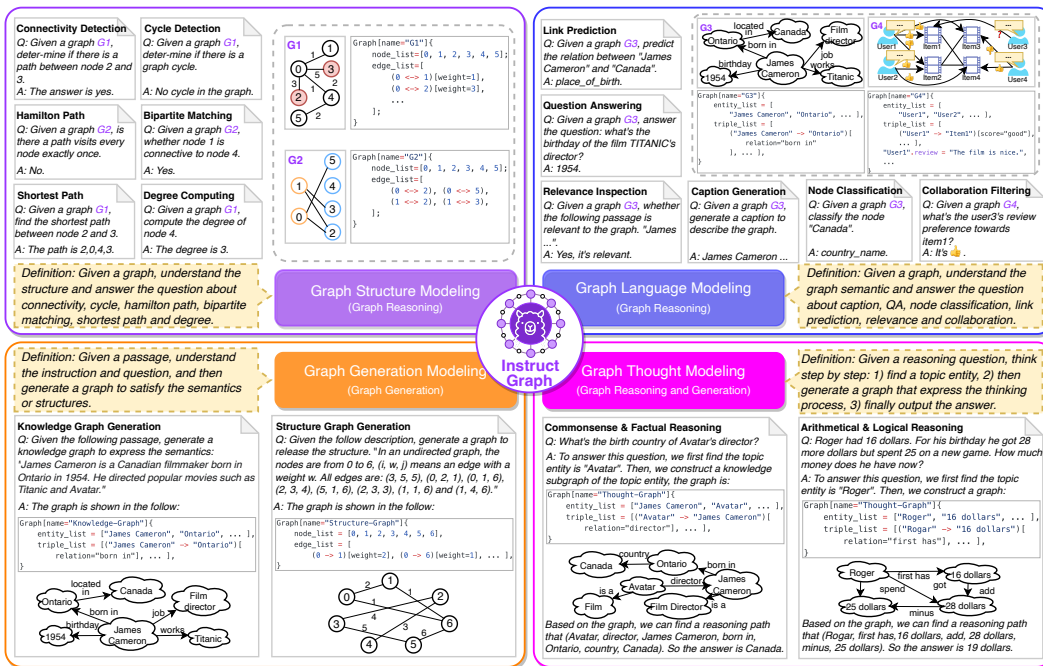


图 1: 原文 Figure 1: 四大类以图为中心的推理与生成任务示例（图内英文为原文保留）。左上为图结构建模（连通/环/最短路等），右上为图语言建模（图问答/节点分类/链接预测等），左下为图生成建模（知识图谱/结构图生成），右下为图思维建模（把推理过程画成思维图再作答）。注意每张图都被写成 node_list/edge_list/triple_list 这样的“类代码”格式。

数据规模：作者一共收集/构造了 **29 个图任务**，组成约 **160 万条指令**微调数据和约 **10 万条**偏好对齐数据（精确统计见原文 Table 10：指令训练集约 134 万、偏好训练集约 8.6 万）。数据来源很广，覆盖 NLGraph（图结构）、WebNLG/GenWiki（图标题）、WebQSP/GraILQA（图问答）、Cora/Citeseer/Pubmed/Arxiv/Products（节点分类）、Wikidata/FB15K-237/ConceptNet（链接预测）、UIE（信息抽取/知识图谱生成）等经典数据集。

【小白补丁】“图思维建模”为什么只用来评测、不参与训练

作者只用前三类任务做指令微调；第四类“图思维建模”（让模型把算术/逻辑题的推理过程画成一张思维图）故意不放进训练集，专门留作测试。这样可以检验：在前三类图任务上学到的能力，能不能迁移去帮助原本没训练过的推理任务——这是对“图能力是否真正泛化”的一次考验。

6 六、方法逐节精讲

InstructGraph 的整体骨架（图 2）由三个模块串成一条流水线：图输入工程（Graph Input Engineering）→ 图指令微调（Graph Instruction Tuning）→ 图偏好对齐（Graph Preference Aligning）。

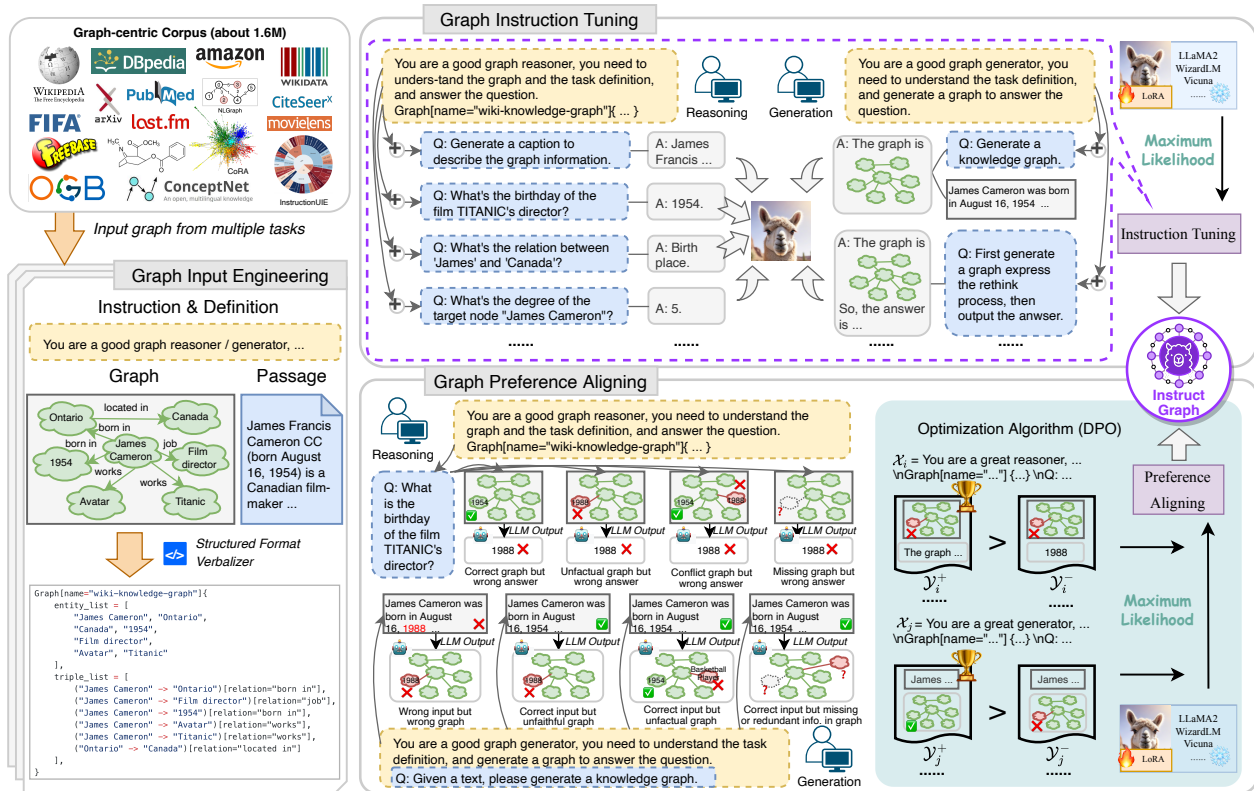


图 2: InstructGraph 总体框架 (原文 Figure 2, 图内英文为原文保留): 左下「Graph Input Engineering」——用结构化格式表述器 (Verbalizer) 把图写成“类代码”格式; 上方「Graph Instruction Tuning」——把约 160 万条图任务数据按推理/生成喂给 LLM, 用最大似然 (Maximum Likelihood) 做指令微调, 仅训练 LoRA 适配器 (图中 ♡ 表示可训练、* 表示冻结主干); 右下「Graph Preference Aligning」——构造四种“图幻觉”场景的错误答案作为负样本, 用 DPO 优化算法把模型推向更可靠的输出。

6.1 第 1 步: 图输入工程——结构化格式表述器

这是本文第一块技术核心, 回答“怎么把图对齐到文本、塞进 LLM 的序列接口”。做法已在第四节讲过: 用表述器 $M(\cdot)$ 把图 G_i 映射成“类代码”序列 $C_i = M(G_i)$ 。基本格式是: 所有节点 (或实体) 列进 `node_list` (或 `entity_list`), 所有边 (或三元组) 列进 `edge_list` (或 `triple_list`); 带附加信息的节点用“面向对象”写法表达, 如 `User1.review=The film is nice.`。关键好处: 这种格式不丢图的任何结构, 又能被 LLM 轻松读写、被程序解析回真图。

6.2 第 2 步: 图指令微调 (得到 InstructGraph-INS)

经过图输入工程后, 每条样本都变成了纯文本序列, 于是可以直接复用最标准的因果语言建模 (CLM) 目标继续训练 LLM——不需要任何特殊网络结构。对任务数据集 $D_j = \{(I_i, G_i, P_i, A_i)\}_{i=1}^{N_j}$,

用**最大似然优化**，让模型在给定输入 X_i 时尽量输出参考答案 A_i ：

$$\mathcal{L}(D_j) = - \sum_{i=1}^{N_j} \log \pi_{\theta}(Y_i = A_i | X_i). \quad (1)$$

其中 π_{θ} 是参数为 θ 的 LLM， Y_i 是模型输出， X_i 是输入序列， A_i 是参考答案。不同任务的输入/输出拼法不同（见原文 Table 1）：比如图结构任务 $X_i = [I_i, C_i]$ 、 $Y_i = A_i$ ；图生成任务 $X_i = [I_i, P_i]$ 、 $Y_i = C_i$ （即让模型生成那段类代码图）。训练完得到 **InstructGraph-INS**。

【小白补丁】怎么读公式 (1)：它其实就是“背标准答案”

$\log \pi_{\theta}(Y_i = A_i | X_i)$ 表示“在输入 X_i 下，模型给出正确答案 A_i 的对数概率”。前面加负号、再求和，就是让这些概率尽量大（损失尽量小）——也就是逼着模型在每个图任务上都尽量输出对的答案。这是所有指令微调最朴素的写法，本文的创新不在这个公式，而在于喂进去的 X_i 是统一的“类代码”图格式。

6.3 第 3 步：图偏好对齐——用 DPO 压幻觉（得到 InstructGraph-PRE）

这是本文第二块技术核心。光会做题还不够，模型有时会**幻觉**：图里缺信息时硬编答案、或生成一张事实错误的图。作者为图推理和图生成分别设计了**多种幻觉场景**，并**自动负采样**造出“坏答案 Y_i^- ”：

- **图推理的四种幻觉**：图正确但答错；不实图（unfactual，与外部知识矛盾）导致答错；冲突图（conflict，图内信息自相矛盾）导致答错；缺失图（missing，缺关键信息）导致答错。
- **造负样本的手法**：对场景 S ，从别的样本里随机抓一个答案当 Y_i^- ；对其余场景，随机**替换/增加/删除**图里的一些节点或边，构造出“被改坏的图”。此时**原来的正确答案反而成了负样本 Y_i^-** ，而正样本 Y_i^+ 是一句话：“抱歉，输入图含有错误信息，所以该问题无法直接回答。”——即教模型学会“图不对就拒答/纠错”。
- **图生成的幻觉**：图生成更难（要输出一整段完整准确的类代码）。同样用替换/增删操作直接造出错误图当 Y_i^- ，原始正确图作为 Y_i^+ 。

有了成对的 (X_i, Y_i^+) 和 (X_i, Y_i^-) ，先按 **Bradley-Terry** 偏好模型定义“偏好概率”：

$$p_{\theta}(Y_i^+ > Y_i^- | X_i) = \frac{1}{1 + \exp\{r(Y_i^+, Y_i^-, X_i)\}}, \quad (2)$$

$$r(Y_i^+, Y_i^-, X_i) = -\beta \log \frac{\pi_{\theta}(Y_i^+ | X_i)}{\pi_{\text{ref}}(Y_i^+ | X_i)} + \beta \log \frac{\pi_{\theta}(Y_i^- | X_i)}{\pi_{\text{ref}}(Y_i^- | X_i)}.$$

其中 β 是平衡因子， π_{θ} 是待训练的**策略模型**、 π_{ref} 是固定的**参考模型**（都用上一步的 InstructGraph-INS 初始化）。最终用最大似然优化 DPO 目标：

$$\mathcal{J}(\pi_{\theta}, \pi_{\text{ref}}) = -\mathbb{E}_{(X_i, Y_i^+, Y_i^-) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(Y_i^+ | X_i)}{\pi_{\text{ref}}(Y_i^+ | X_i)} - \beta \log \frac{\pi_{\theta}(Y_i^- | X_i)}{\pi_{\text{ref}}(Y_i^- | X_i)} \right) \right]. \quad (3)$$

训练得到的策略 π_θ 即 **InstructGraph-PRE**。

【小白补丁】怎么读 DPO 目标 (3)：一句话拆给你听

看括号里那两项之差： $\beta \log \frac{\pi_\theta(Y^+)}{\pi_{\text{ref}}(Y^+)}$ 衡量“相比初始模型，现在更愿意给好答案吗”，后一项是“更愿意给坏答案吗”。两者相减再过 σ (Sigmoid) 取对数：当模型**把好答案概率提上去、坏答案概率压下去**时，括号里为正、 $\log \sigma$ 接近 0 (损失小)；反之损失大。所以最小化 $\mathcal{J}$ 就等价于“相对参考模型，多给 Y^+ 、少给 Y^- ”。 π_{ref} 像一根锚，防止模型为了讨好偏好而跑偏、忘掉原有能力。而 β 控制“可以偏离锚多远”。这正是 DPO 不需要奖励模型、不跑强化学习就能做偏好对齐的精髓。

训练细节：默认主干为 LLaMA2-7B-HF，最大长度 2048，优化器 AdamW；指令微调阶段学习率 $5e-5$ 、偏好对齐阶段降到 $5e-7$ 。为省显存用了 FSDP + CPU Offloading + FlashAttention + BFloat16，并用 **LoRA** (rank= 32, lora_α = 128) 做参数高效微调——**只训练很少的参数，冻结 LLM 主干**。

7 七、实验与结果解读

对比对象：同规模开源模型 LLaMA2-7B、Vicuna-7B；以及强基线 GPT-3.5 (turbo) 和 GPT-4。评测主要在**零样本 (zero-shot)** 下进行。

7.1 主结果一：图指令任务 (超过 GPT-4)

原文 Table 2 在 29 个图任务上对比，InstructGraph-INS 的**总平均分 79.84%**，比 GPT-4 的 66.76% 高出 **13.08%**，比同规模 LLaMA2 (41.65%) 高出约 38%。下面节选几行最有代表性的数字：

表 2: 图指令任务主结果（%，数值源自原文 Table 2，越高越好；本表为节选 + 总平均，加粗为该行最优）。

任务	指标	GPT-3.5	GPT-4	LLaMA2	Vicuna	InstructGraph-INS
环检测 Cycle Dect.	ACC	59.02	61.44	50.79	52.88	91.10
二分图匹配 Bipt. Match	ACC	50.23	66.11	0.00	0.00	76.36
最短路径 Shrt. Path	ACC	38.99	49.03	0.00	0.00	66.29
图问答 PathQSP	EM	52.54	68.64	42.70	31.90	86.40
图问答 WebQSP	EM	53.73	61.57	40.07	26.42	73.30
链接预测 Wikidata	Hits@1	43.73	62.94	10.75	10.38	96.52
链接预测 FB15K-237	Hits@1	60.34	66.88	0.00	0.00	98.91
推荐 Amazon	Hits@1	27.09	59.77	44.40	16.40	78.80
信息抽取 UIE	F1	24.41	26.22	20.21	26.11	76.82
总平均 Avg.		59.45	66.76	41.65	46.06	79.84

怎么读这张表：(1) 在“图结构”类任务上，LLaMA2/Vicuna 经常考 **0 分**（完全不会做二分图匹配、最短路），而 InstructGraph 普遍 60%~90%；说明这些结构推理能力确实是被专门训练出来的。(2) 在链接预测上 InstructGraph 高达 96%~99%，远超 GPT-4，体现“类代码三元组格式”对关系预测的天然契合。(3) 作者也坦诚：在 Degree Computing、WebNLG、GenWiki、WikiTQ、Citeseer 等少数任务上 InstructGraph 略逊于 GPT-3.5/GPT-4，因为超大参数模型“记”住了更多相似知识；但在其余推理任务上 InstructGraph 仍领先约 10%。

7.2 主结果二：图偏好任务（DPO 进一步压幻觉）

作者构造了一个“幻觉测试集”：每条样本含一个对答案和一个错答案，计算 LLM 在两者上的**困惑度（PPL）**，选 PPL 更低（更“顺”）的那个作为模型的偏好结果，于是**准确率越高 = 越能识别正确答案 = 幻觉越少**。

表 3: 图偏好任务主结果（%，数值源自原文 Table 3，越高越好/越忠实；加粗为最优）。

方法 (7B)	是否对齐	结构	标题	图问答	节点分类	信息抽取
LLaMA2	否	38.64	57.96	70.70	74.68	37.40
Vicuna	否	39.12	62.37	64.38	77.63	40.80
InstructGraph-INS	否	50.32	81.15	77.85	83.16	69.14
InstructGraph-PRE	是	57.80	87.44	84.44	88.98	91.44

解读：仅做指令微调的 InstructGraph-INS（平均 72.32%）就已经比 LLaMA2/Vicuna 高约 16%、15%，说明**把任务知识注入参数本身就能减少幻觉**。在此之上再叠加 DPO 偏好对齐，InstructGraph-

PRE 平均再涨约 10%（达 82.02%），尤其在信息抽取上从 69.14 飙到 91.44——证明**精心设计的偏好优化能进一步逼近上限、教会模型“图不对就别硬答”**。

7.3 其他关键发现

- **“类代码”格式 vs 传统模板**（原文 Table 7）：把图写成结构化代码，全面优于 InstructGLM 式的自然语言模板；且**传统模板根本无法支持图生成**，而类代码格式可以。这直接验证了核心设计的价值。
- **消融实验**（原文 Table 8）：去掉任一类建模任务都会掉分——图语言建模对“图问答/节点分类”最关键，图生成建模对“信息抽取”最关键；在偏好侧，**去掉“缺失图”负样本掉分最多**，说明图里缺关键信息是幻觉的最大来源。
- **参数高效微调**（原文 Figure 4）：在六种参数规模下对比 LoRA、Prefix-tuning、Adapter、全量微调，**LoRA 用极少可训练参数就逼近全量微调**，性价比最高。
- **换骨干/换规模**（原文 Figure 5）：在 LLaMA2-7B/13B、Vicuna-7B/13B 上 InstructGraph 都能带来大幅提升，且更大模型收益更大。有趣的是：微调前 Vicuna 略强于 LLaMA2，但**图指令微调后这个优势被反转**，作者分析 LLaMA2 是更好的“起点”骨干。
- **思维规划迁移**（原文 Table 5）：在算术/符号/机器人/逻辑等未训练的推理任务上，用 CoT 或“图思维建模（GTM）”提示，InstructGraph-INS 都优于 LLaMA2/Vicuna，说明图能力能迁移去帮助通用推理。
- **不伤通用能力**（原文 Table 4、Table 6）：在 HaluEval、TruthfulQA、BBH、MMLU 等通用 NLP 基准上，InstructGraph 仍与同规模开源模型持平或略优，说明**专攻图任务没有把通用语言能力练废**。

8 八、小白知识点卡片（速查）

- **结构化格式表述器（structured format verbalizer）**：本文核心，把任意图写成统一“类代码”（entity_list/triple_list）的转换器，不外挂图编码器。
- **指令微调（instruction tuning）**：用“指令 + 输入 + 答案”样本继续训练 LLM，让它学会按指令做任务。
- **DPO（直接偏好优化）**：给一对“好/坏答案”，不训练奖励模型、不跑强化学习，直接把模型推向好答案的偏好对齐方法。
- **Bradley-Terry 模型**：把“A 比 B 更好”的偏好建模成一个概率的经典统计模型，是 DPO 的理论基础。

- **图幻觉 (graph hallucination)**: 模型在图任务上编造不存在的节点/边, 或图缺信息时硬答。
- **负采样 (negative sampling)**: 本文靠“随机替换/增删图中节点或边”自动造出错误答案当负样本, 省去人工标注。
- **LoRA / PEFT**: 参数高效微调; LoRA 只训练低秩增量矩阵, 冻结主干、省显存。
- **因果语言建模 (CLM)**: 最标准的“预测下一个词”训练目标。
- **Hits@1 / EM / PPL**: 分别为“排第一是否命中”“完全匹配率”“困惑度 (越低越顺)”。

9 九、一句话总结与“图 × 后训练”谱系定位

一句话: InstructGraph 不给 LLM 外挂任何图编码器, 而是用一种统一的“类代码”结构化格式把所有图写出来, 先做**图指令微调**让模型学会 29 个图任务, 再用 **DPO 偏好对齐** (靠“故意改错图”自动造负样本) 压低图幻觉, 最终在图推理与图生成上超过 *GPT-4* 与 *LLaMA2*。

在“图 × 后训练”谱系里的位置: 本系列各篇的关键区别, 就在于“把图的信息塞进 LLM 的方式”不同——G-Retriever 走“检索 + 软提示 (PCST 抠子图)”, LLaGA 走“投影器对齐 (把图向量投影进词空间)”, G1 走“强化学习”。而 InstructGraph 走的是“**纯文本化 (类代码 verbalizer) + 指令微调 + DPO 偏好对齐**”路线: 它不引入任何图神经网络, 把图问题彻底转化成 LLM 最擅长的“读写代码 + 跟随指令”问题, 再用偏好对齐这一步专门治图任务的幻觉。可以把它和 GraphWiz 放在一起看——两者都属于“**指令微调 + 偏好/DPO**”这一支, 代表了“不改 LLM 结构、只靠数据与后训练把图能力灌进去”的思路。

动手复现: 官方代码 <https://github.com/wjn1996/InstructGraph> (含结构化格式表述器、29 个图任务的语料构造、指令微调与 DPO 偏好对齐流程), 是想理解“图 × 指令微调 × DPO”这条路线很好的练手项目。